

## Distribution of Food Orders

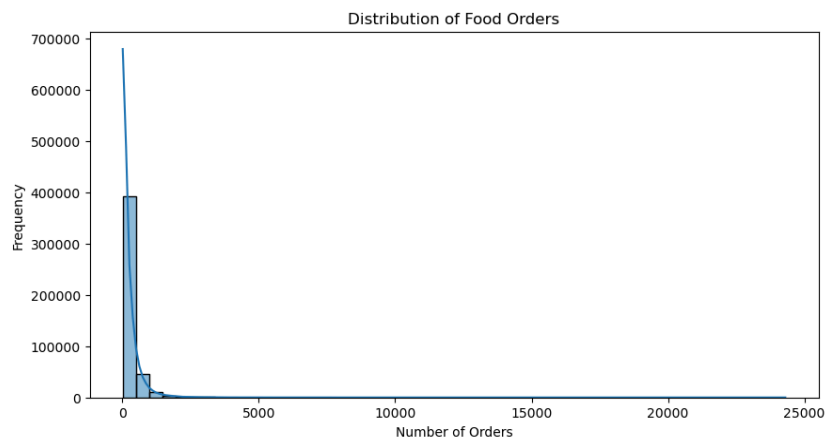
### Description:

The Distribution of Food Orders graph shows how food orders are distributed across the dataset. Most orders are concentrated in the lower range, while only a small number of observations have very high order values. This indicates that the dataset contains a positively skewed distribution, with a few unusually large order values.

### Key Insight:

Most food orders are relatively small, while extreme high-order values occur less frequently.

```
[49]: plt.figure(figsize=(10, 5))  
  
sns.histplot(df["num_orders"], bins=50, kde=True)  
  
plt.title("Distribution of Food Orders")  
plt.xlabel("Number of Orders")  
plt.ylabel("Frequency")  
plt.show()
```



## Weekly Food Demand

### Description:

The Weekly Food Demand graph represents the variation in food orders across different weeks. The demand fluctuates throughout the period, with several peaks and drops. The graph shows that food demand is not constant and changes from week to week.

### Key Insight:

Weekly demand shows noticeable fluctuations, which may be influenced by customer preferences, promotions, weekends, or other business factors.

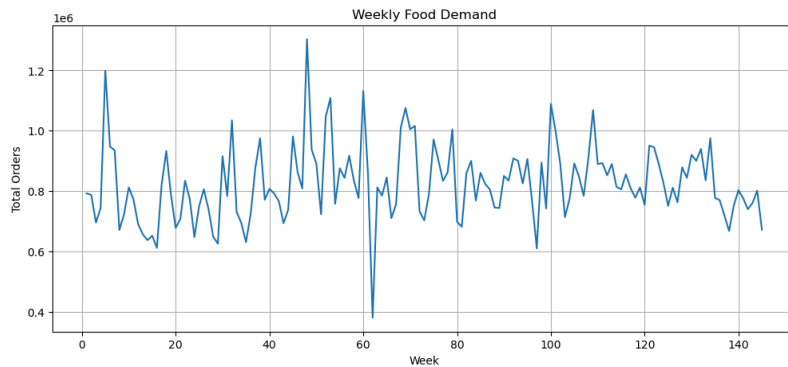
```
[51]: weekly_orders = df.groupby("week")["num_orders"].sum()

plt.figure(figsize=(12, 5))

plt.plot(weekly_orders.index, weekly_orders.values)

plt.title("Weekly Food Demand")
plt.xlabel("Week")
plt.ylabel("Total Orders")
plt.grid(True)

plt.show()
```



## Effect of Email Promotion on Orders

### Description:

This graph compares the average number of orders for customers who received email promotions with those who did not. The average number of orders is considerably higher for the group exposed to email promotions.

### Key Insight:

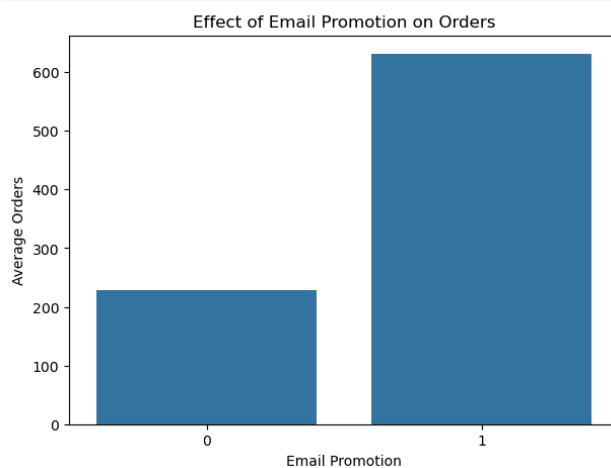
The graph suggests that email promotions have a positive relationship with the number of food orders. Promotional emails may encourage customers to place more orders.

```
[55]: plt.figure(figsize=(7, 5))

sns.barplot(
    x=promotion_orders.index,
    y=promotion_orders.values
)

plt.title("Effect of Email Promotion on Orders")
plt.xlabel("Email Promotion")
plt.ylabel("Average Orders")

plt.show()
```



## Effect of Homepage Feature on Orders

### Description:

This graph compares average orders based on whether a food item or offer was featured on the homepage. The average number of orders is higher for items that were featured on the homepage.

### Key Insight:

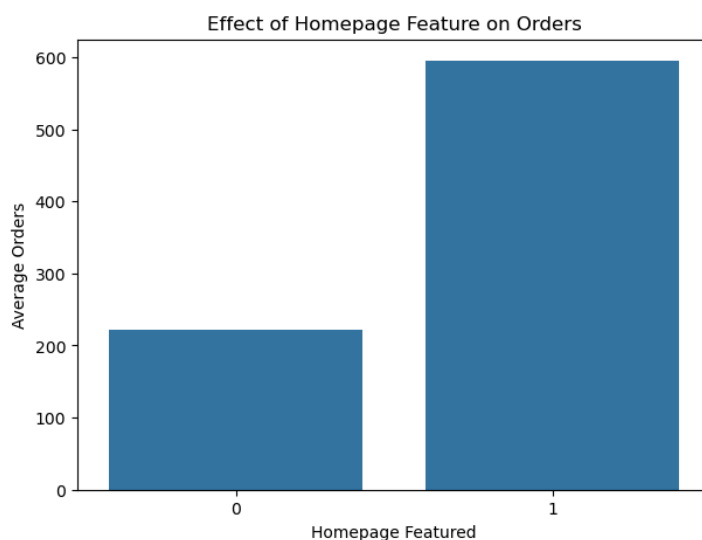
Homepage visibility appears to have a positive effect on orders. Featuring products prominently may increase customer attention and encourage more purchases.

```
[59]: plt.figure(figsize=(7, 5))

sns.barplot(
    x=homepage_orders.index,
    y=homepage_orders.values
)

plt.title("Effect of Homepage Feature on Orders")
plt.xlabel("Homepage Featured")
plt.ylabel("Average Orders")

plt.show()
```



## Average Orders by Cuisine

### Description:

The Average Orders by Cuisine graph compares the average number of orders across different cuisine categories. Among the displayed cuisines, Italian cuisine has the highest average orders, followed by Thai and Indian cuisine. Continental cuisine has the lowest average order level among the categories shown.

### Key Insight:

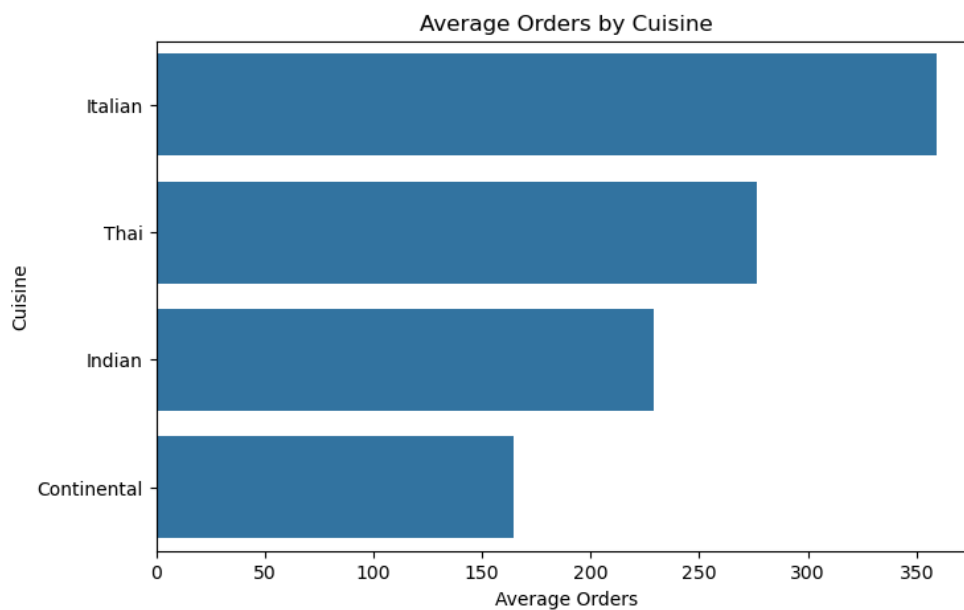
Cuisine type has a noticeable relationship with order volume, with Italian cuisine showing the strongest customer demand in this dataset.

```
[64]: plt.figure(figsize=(8, 5))

sns.barplot(
    x=cuisine_orders.values,
    y=cuisine_orders.index
)

plt.title("Average Orders by Cuisine")
plt.xlabel("Average Orders")
plt.ylabel("Cuisine")

plt.show()
```



# Average Orders by Food Category

## Description:

This horizontal bar graph compares the average number of orders for different food categories. The categories show considerable variation in average order volume. Pizza and beverages are among the categories with relatively high average orders, while several other categories have comparatively lower demand.

## Key Insight:

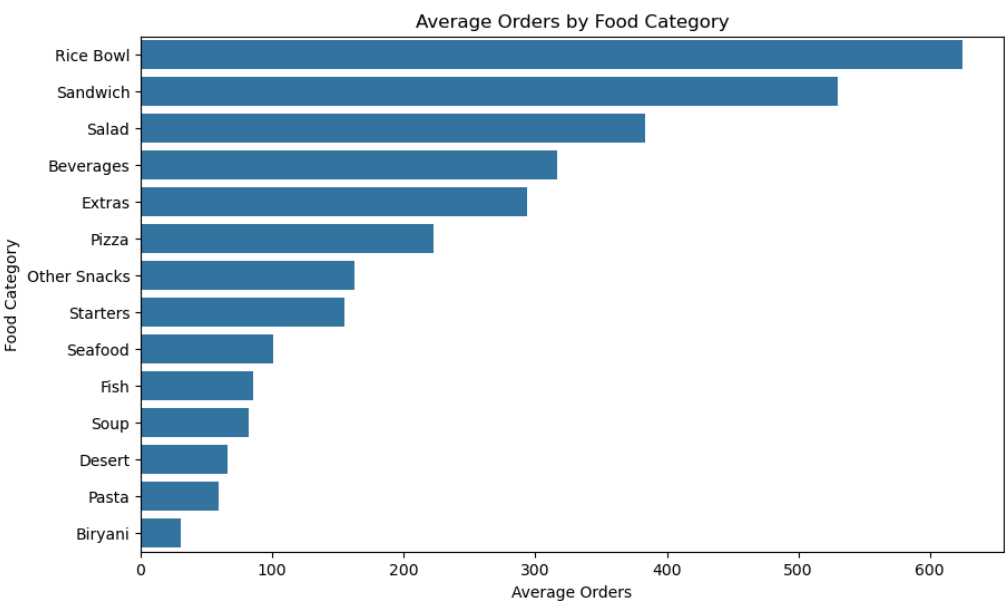
Customer demand differs significantly across food categories. High-performing categories can be given greater attention for promotions, inventory planning, and business strategy.

```
[68]: plt.figure(figsize=(10, 6))

sns.barplot(
    x=category_orders.values,
    y=category_orders.index
)

plt.title("Average Orders by Food Category")
plt.xlabel("Average Orders")
plt.ylabel("Food Category")

plt.show()
```



## Checkout Price vs Number of Orders

### Description:

The scatter plot illustrates the relationship between checkout price and the number of orders. Most observations are concentrated around lower checkout prices and lower order counts. As the checkout price increases, the number of observations generally decreases.

### Key Insight:

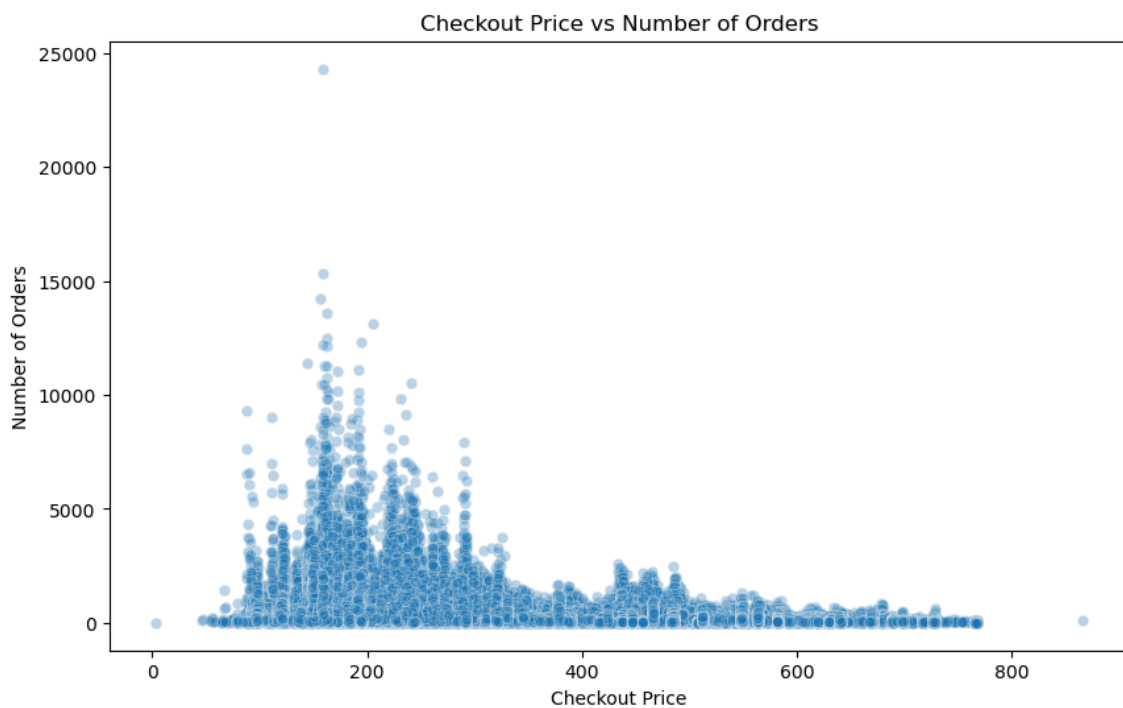
The graph indicates that very high checkout prices are associated with fewer observations and generally lower order volumes. However, the relationship is not perfectly linear.

```
[70]: plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="checkout_price",
    y="num_orders",
    alpha=0.3
)

plt.title("Checkout Price vs Number of Orders")
plt.xlabel("Checkout Price")
plt.ylabel("Number of Orders")

plt.show()
```



## Base Price vs Number of Orders

### Description:

This scatter plot shows the relationship between the base price of food items and the number of orders. The observations are widely distributed, with a higher concentration of orders around certain price ranges. Extremely high order counts are less frequent.

### Key Insight:

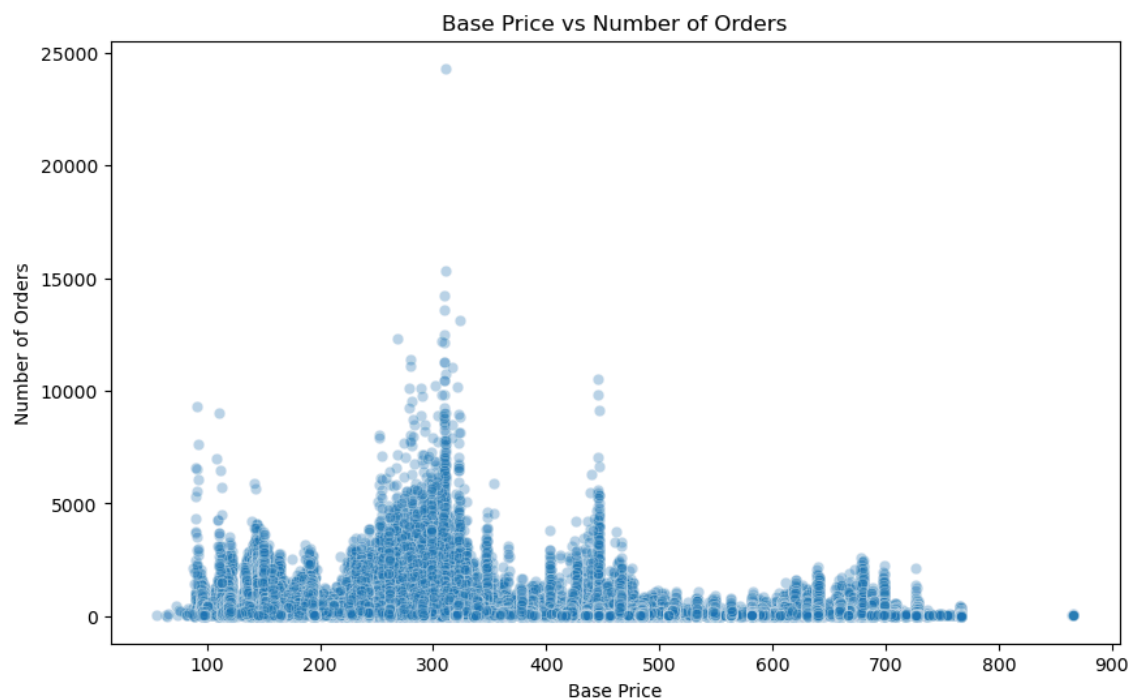
Base price appears to influence order behavior, but the relationship is affected by other variables as well. Price alone cannot completely explain the variation in the number of orders.

```
[73]: plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="base_price",
    y="num_orders",
    alpha=0.3
)

plt.title("Base Price vs Number of Orders")
plt.xlabel("Base Price")
plt.ylabel("Number of Orders")

plt.show()
```



# Correlation Matrix

## Description:

The correlation matrix represents the relationships between different numerical and encoded features in the dataset. The values range from negative to positive correlation. Stronger positive correlations indicate that two variables tend to increase together, while negative correlations indicate an inverse relationship.

## Key Insight:

The correlation matrix helps identify important relationships between variables and can be used to understand which features may contribute to predicting food orders.

```
[75]: numeric_cols = [
    "week",
    "checkout_price",
    "base_price",
    "emailer_for_promotion",
    "homepage_featured",
    "num_orders",
    "city_code",
    "region_code",
    "op_area"
]

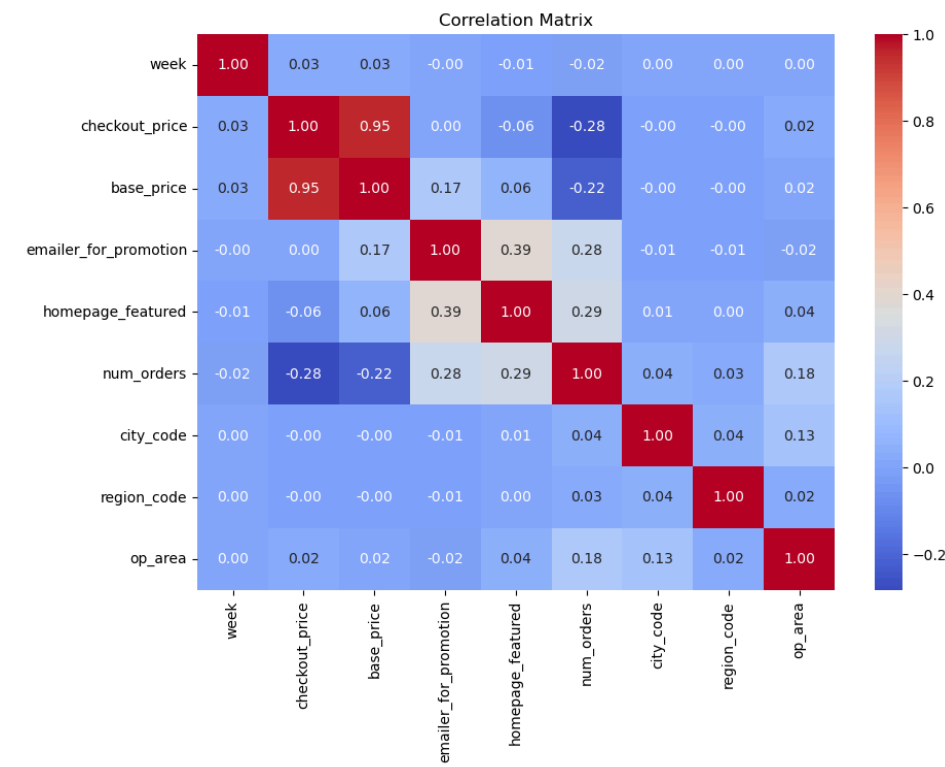
correlation = df[numeric_cols].corr()

plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix")

plt.show()
```



## Actual vs Predicted Food Orders – Random Forest

### Description:

This scatter plot compares the actual food order values with the values predicted by the Random Forest model. Each point represents an observation, with actual orders shown on the x-axis and predicted orders on the y-axis.

### Key Insight:

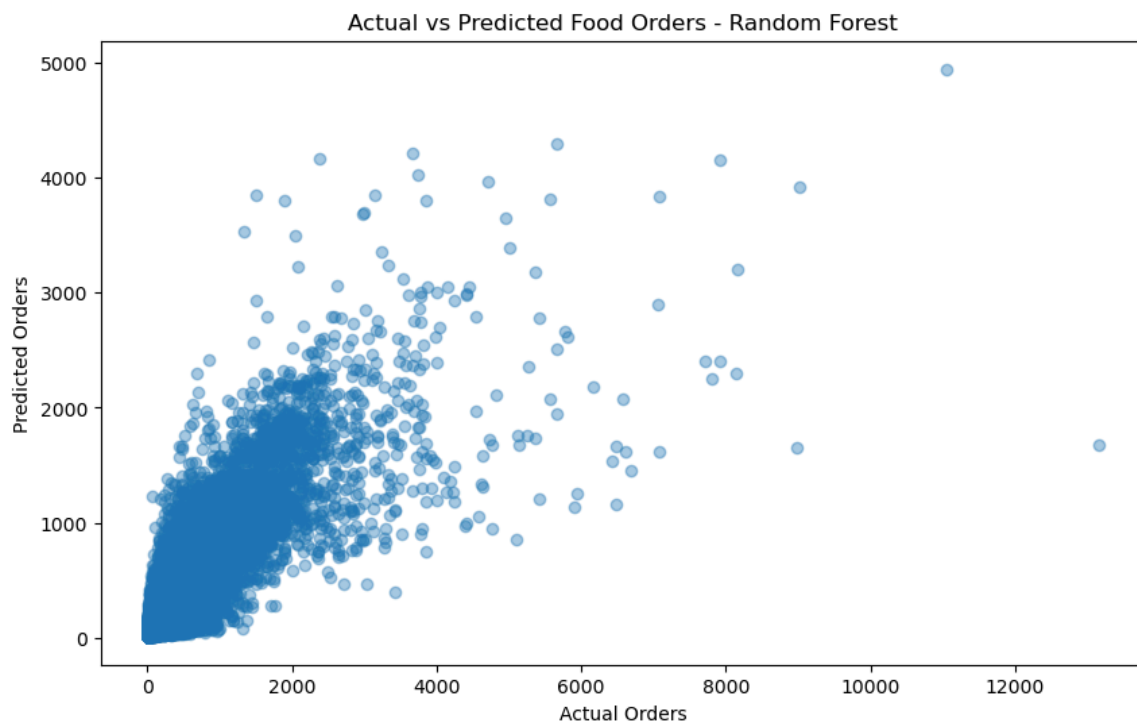
The points show a positive relationship between actual and predicted orders. The model is able to capture the general pattern of the target variable, although some predictions differ considerably from the actual values.

```
[133]: plt.figure(figsize=(10, 6))

plt.scatter(
    y_val,
    pred_rf,
    alpha=0.4
)

plt.xlabel("Actual Orders")
plt.ylabel("Predicted Orders")
plt.title("Actual vs Predicted Food Orders - Random Forest")

plt.show()
```



## Actual vs Predicted Food Orders with Reference Line

### Description:

This graph compares actual food orders with Random Forest predicted orders and includes a reference diagonal line. Ideally, predictions should be close to this line because points near the line represent predictions close to the actual values.

### Key Insight:

Many observations are concentrated below the reference line, indicating that the model tends to underestimate some higher order values. The graph also shows that prediction accuracy decreases for extreme order values.

```
[135]: plt.figure(figsize=(10, 6))

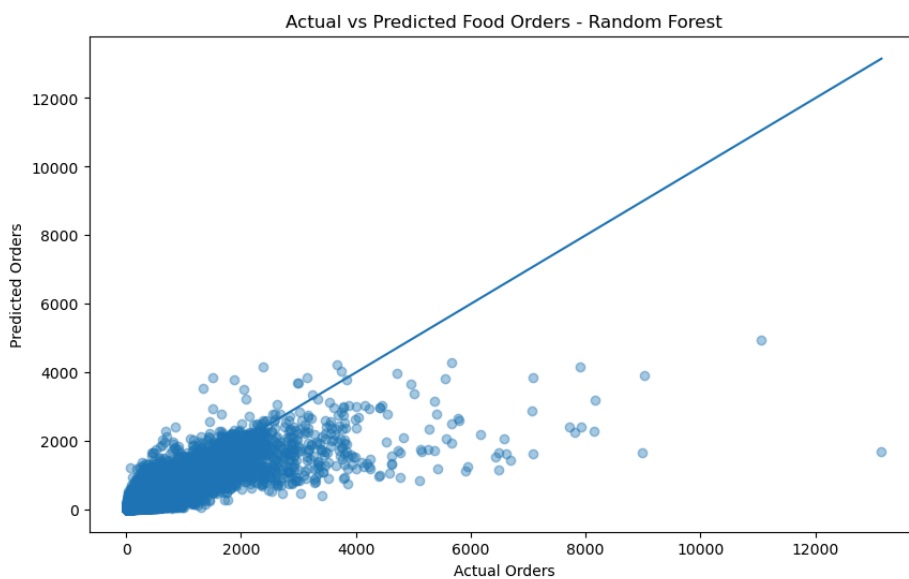
plt.scatter(
    y_val,
    pred_rf,
    alpha=0.4
)

min_value = min(y_val.min(), pred_rf.min())
max_value = max(y_val.max(), pred_rf.max())

plt.plot(
    [min_value, max_value],
    [min_value, max_value]
)

plt.xlabel("Actual Orders")
plt.ylabel("Predicted Orders")
plt.title("Actual vs Predicted Food Orders - Random Forest")

plt.show()
```



## Random Forest Feature Importance

### Description:

The Feature Importance graph shows the relative contribution of different features used by the Random Forest model. Features with larger importance values have a greater influence on the model's predictions.

The graph shows **checkout\_price** as the most important feature, followed by **category\_encoded**. Other influential features include **cuisine\_encoded**, **op\_area**, **base\_price**, **week**, **meal\_id**, **center\_id**, and other variables.

### Key Insight:

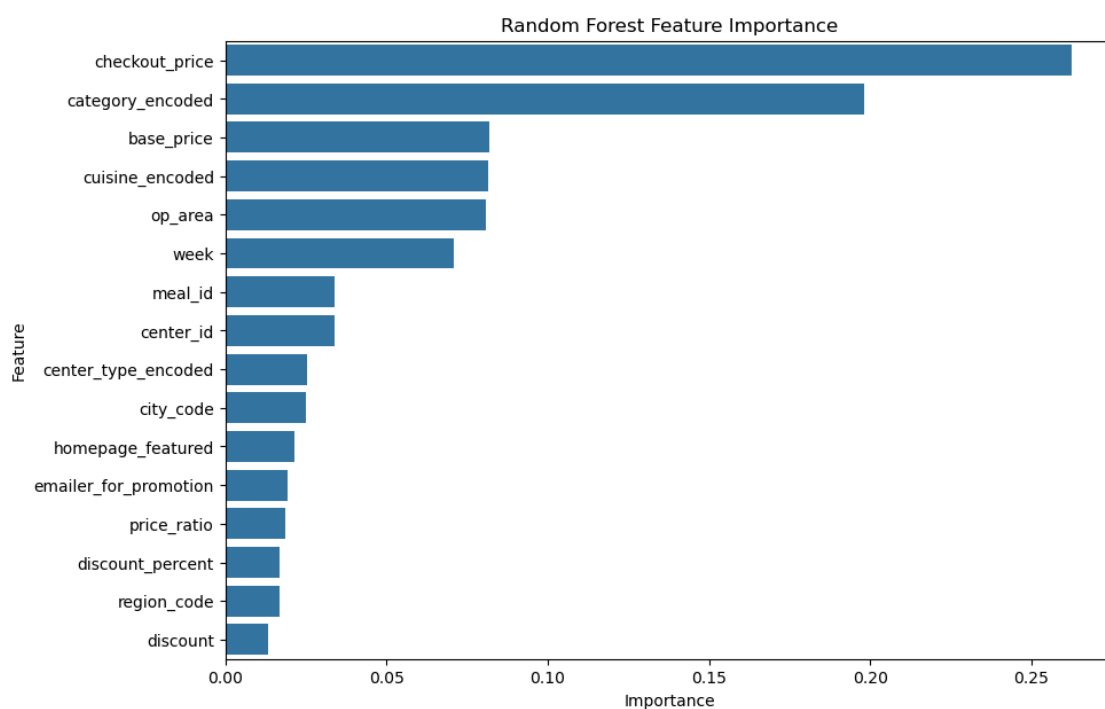
Checkout price is the most influential feature in predicting food orders according to the Random Forest model.

```
[139]: plt.figure(figsize=(10, 7))

sns.barplot(
    data=importance,
    x="Importance",
    y="Feature"
)

plt.title("Random Forest Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()
```



# Model Performance Comparison – R² Score

## Description:

This bar graph compares the performance of three machine learning models using the R² score: Linear Regression, Random Forest, and XGBoost. A higher R² score indicates that the model explains a greater proportion of the variation in the target variable.

The graph shows that **Random Forest and XGBoost perform considerably better than Linear Regression**.

## Key Insight:

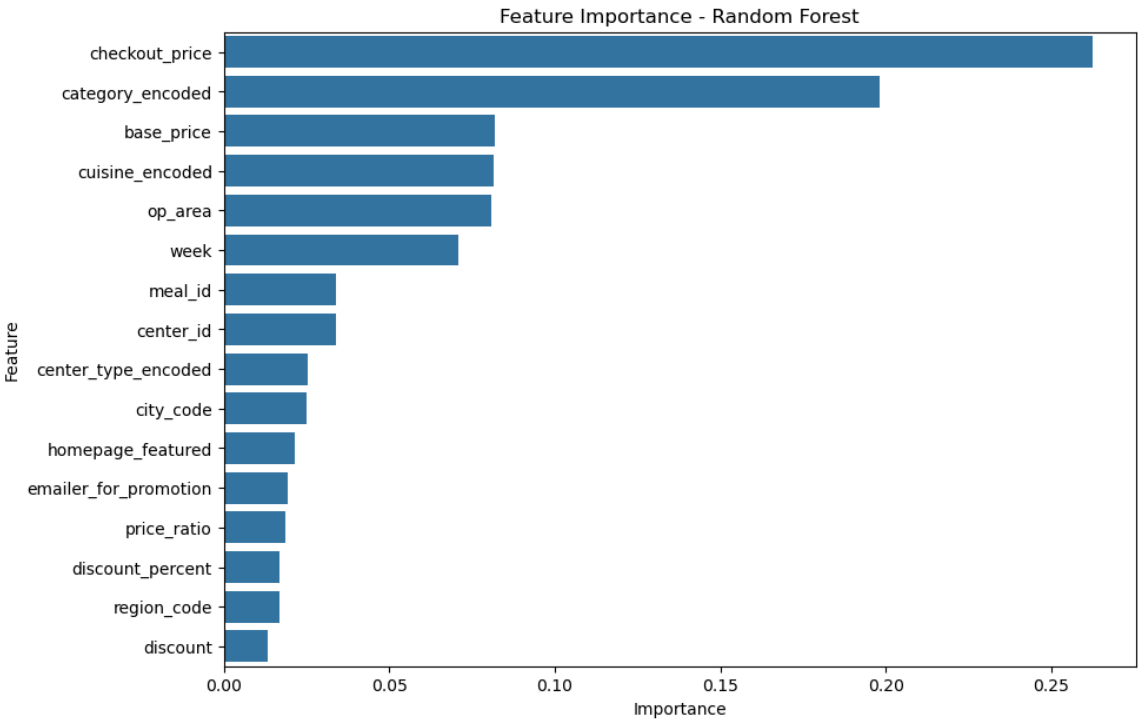
Random Forest achieves the highest R² score among the models shown, indicating that it provides the strongest predictive performance for this dataset among the compared models.

```
plt.figure(figsize=(10, 7))

sns.barplot(
    data=importance,
    x="Importance",
    y="Feature"
)

plt.title("Feature Importance - Random Forest")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()
```



# Final Actual vs Predicted Food Orders

## Description:

The final Actual vs Predicted Food Orders graph evaluates the performance of the Random Forest model by comparing actual order values with predicted order values. The diagonal line represents the ideal prediction scenario where predicted values are equal to actual values.

## Key Insight:

The predictions follow the general direction of the actual values, demonstrating that the Random Forest model captures the overall relationship. However, the larger deviations for high order values indicate that there is still room for improving prediction accuracy.

```
plt.figure(figsize=(10, 7))

sns.barplot(
    data=importance,
    x="Importance",
    y="Feature"
)

plt.title("Feature Importance of Random Forest Model")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```

